

Project Alpha Technical Design Document

[Project Alpha Technical Design Document](#)

[Revision History](#)

[1. Introduction](#)

[2. High-Level Architecture](#)

[3. RAG Pipeline Design](#)

[4. Data Design](#)

[5. API Design](#)

[6. Scaling and Deployment](#)

[7. Monitoring and Observability](#)

[7A. Security Architecture](#)

[7B. Caching Strategy](#)

[8. Internal Contacts](#)

[9. Related Documents](#)

[10. References](#)

Project Alpha Technical Design Document

Series Tech Ltd. — Internal Use Only

[Company Logo Placeholder — SeriesTech_Logo.png]

Classification: Internal — Confidential **Document Number:** STL-ALPHA-TDD-005 **Version:** 2.1 **Last Updated:** 2026-04-02 **Owner:** Project Alpha Tech Lead **Approver:** Chief Architect

Revision History

Version	Date	Author	Summary of Changes
1.0	2025-02-10	V. Krishnan (Eng)	Initial architecture proposal
1.5	2025-06-30	V. Krishnan (Eng)	Added retrieval pipeline design
2.0	2025-12-05	A. Deshmukh (Arch)	Added scaling and deployment design
2.1	2026-04-02	A. Deshmukh (Arch)	Updated with reranking service design

1. Introduction

This Technical Design Document describes the system architecture, component design, and data flow for Project Alpha, implementing the requirements defined in the Project Alpha Requirements Specification (STL-ALPHA-REQ-004).

1.1 Design Goals

- Multi-tenant isolation at both the data layer and the retrieval layer

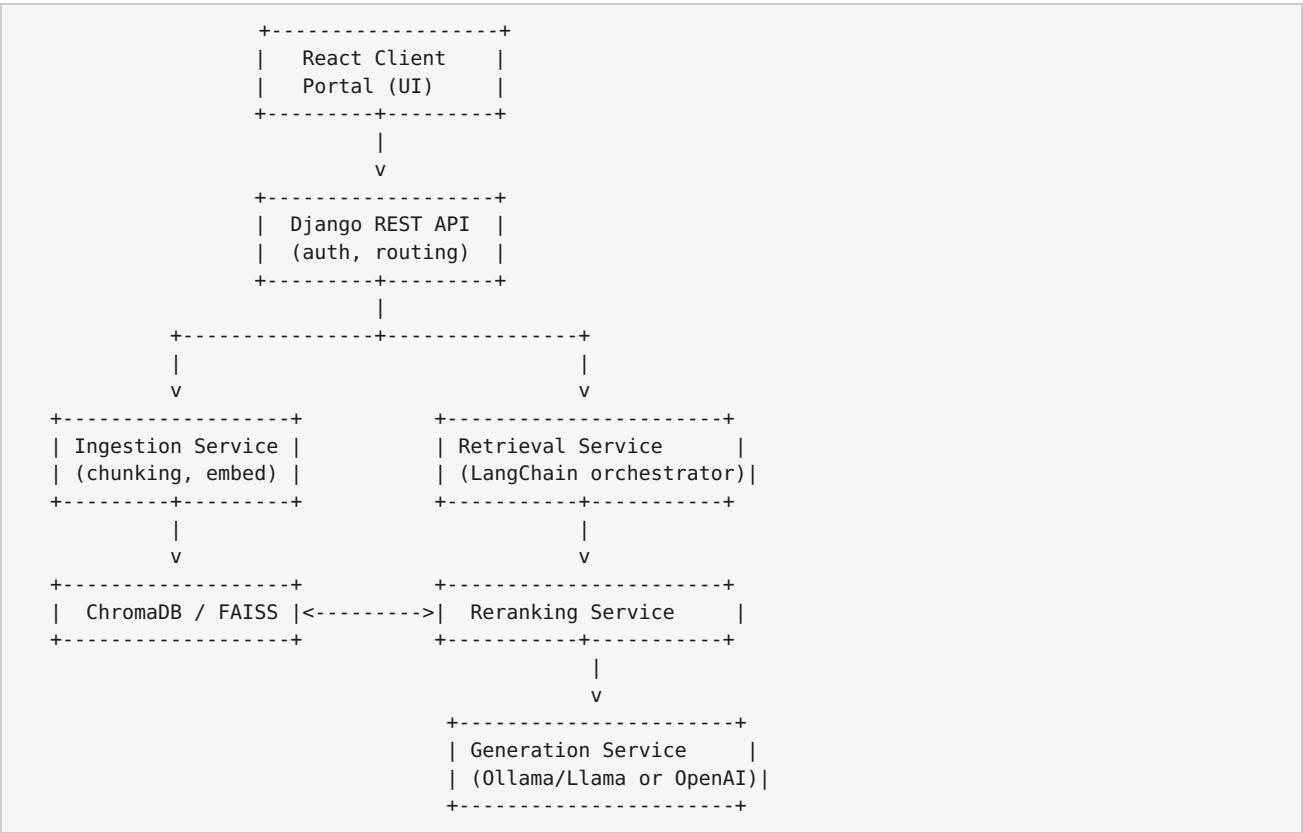
- Pluggable generation backend (Ollama/Llama for on-prem, OpenAI API for cloud-opt-in tenants)
- Horizontal scalability of ingestion and retrieval services independently
- Observability across the full RAG pipeline, from ingestion to answer generation

1.2 Technology Stack

Layer	Technology
Frontend	React (client portal), Angular (internal admin console)
Backend API	Django REST Framework
Orchestration	LangChain
Vector Store	ChromaDB (primary), FAISS (high-volume fallback)
LLM Serving	Ollama (Llama 3 8B/70B), OpenAI API (opt-in tenants)
Relational Store	PostgreSQL
Cache/Queue	Redis
Containerization	Docker, Kubernetes

2. High-Level Architecture

2.1 Component Overview



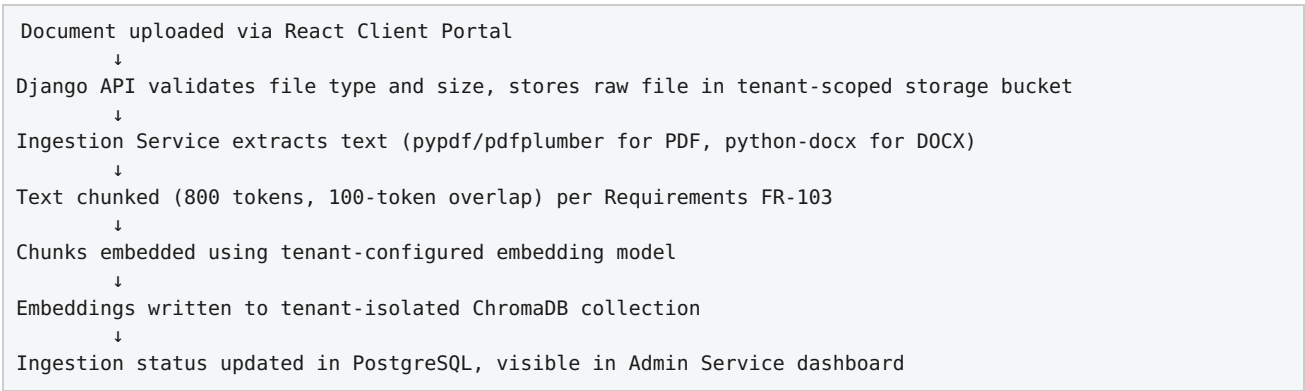
2.2 Service Responsibilities

Service	Responsibility
Ingestion Service	Parses PDF/DOCX/TXT, chunks text, generates embeddings,

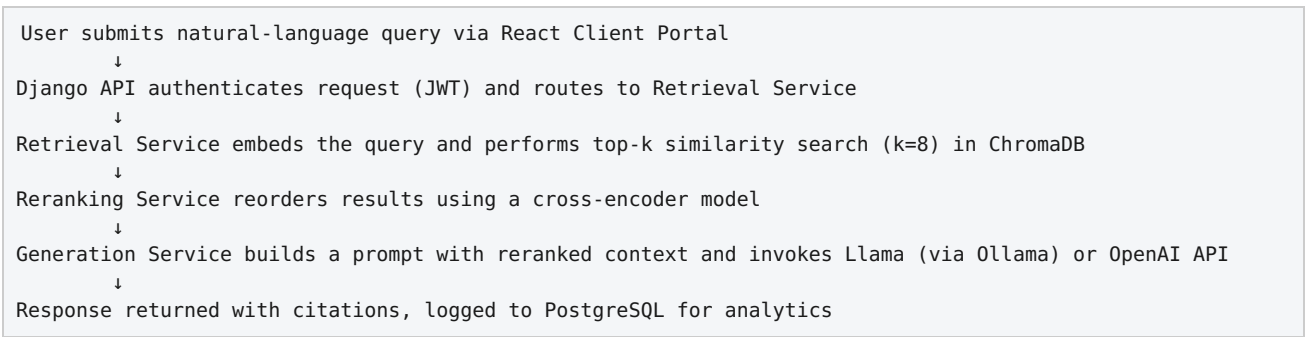
	writes to vector store
Retrieval Service	Accepts queries, performs vector similarity search, invokes reranking
Reranking Service	Applies cross-encoder reranking to top-k retrieved chunks
Generation Service	Constructs the LLM prompt with retrieved context and generates the final answer
Admin Service	Manages tenants, users, roles, and usage analytics

3. RAG Pipeline Design

3.1 Ingestion Flow



3.2 Query Flow



3.3 Embedding and Chunking Configuration

Parameter	Default Value	Configurable Per Tenant
Chunk size	800 tokens	Yes
Chunk overlap	100 tokens	Yes
Embedding model	bge-large-en (self-hosted)	Yes, OpenAI embeddings supported as alternative
Top-k retrieval	8	Yes
Reranker model	bge-reranker-base	Yes

4. Data Design

4.1 Core PostgreSQL Schema (Simplified)

Table	Key Columns	Purpose
tenants	id, name, plan_tier, created_at	Tenant registry
documents	id, tenant_id, filename, format, ingestion_status	Document metadata
document_chunks	id, document_id, chunk_index, token_count	Chunk metadata (embeddings live in ChromaDB, referenced by chunk id)
queries	id, tenant_id, user_id, query_text, response_time_ms	Query analytics
users	id, tenant_id, email, role	Tenant users with RBAC role

4.2 ChromaDB Collection Strategy

Each tenant is assigned a dedicated ChromaDB collection named `tenant_<tenant_id>_docs`, enforcing hard isolation at the collection level in addition to application-layer tenant checks, directly mitigating Risk R-03 in the Requirements Specification’s Risk Register.

5. API Design

5.1 Key Endpoints

Method	Endpoint	Description
POST	<code>/api/v1/tenants/{tenant_id}/documents</code>	Upload and ingest a document
POST	<code>/api/v1/tenants/{tenant_id}/query</code>	Submit a natural-language query
GET	<code>/api/v1/tenants/{tenant_id}/documents/{id}/status</code>	Check ingestion status
GET	<code>/api/v1/tenants/{tenant_id}/analytics</code>	Retrieve usage analytics

These endpoints follow the REST API Development Guidelines defined in Engineering Standards (STL-ENG-STD-003), Section 8, including JWT authentication and standard error envelopes.

6. Scaling and Deployment

6.1 Environment Matrix

Environment	Purpose	Infrastructure
Dev	Developer testing	Docker Compose, local Ollama
Staging	Pre-release validation	Kubernetes (AKS), shared ChromaDB namespace
Production	Live client traffic	Kubernetes (AKS/EKS depending on client region), dedicated ChromaDB per high-tier tenant

6.2 Horizontal Scaling Strategy

Ingestion Service and Retrieval Service are deployed as independently scaled Kubernetes Deployments, allowing ingestion-heavy workloads (bulk document uploads) to scale without affecting query latency for concurrent users, addressing NFR-101 through NFR-104 in the Requirements Specification.

6.3 GPU Sizing for Ollama Deployments

Model	Minimum GPU	Recommended for Production
Llama 3 8B	16 GB VRAM	NVIDIA A10G
Llama 3 70B	80 GB VRAM (quantized: 48 GB)	NVIDIA A100

Note: On-prem tenants using Llama 3 70B without adequate GPU sizing frequently see query latency exceed NFR-102; the Projects team should validate hardware during the client discovery phase.

7. Monitoring and Observability

Project Alpha services emit structured logs and metrics consumed by the shared observability stack described in the Enterprise System Architecture & FAQ (STL-ARCH-SYS-009), Section on Monitoring. Key metrics tracked include retrieval precision, reranking latency, generation latency, and citation accuracy sampled through periodic human review.

7A. Security Architecture

7A.1 Tenant Isolation Controls

Beyond the ChromaDB collection-per-tenant strategy described in Section 4.2, Project Alpha enforces tenant isolation at three additional layers: the PostgreSQL row-level security policies scoping every query to the authenticated tenant_id, the API gateway’s JWT tenant claim validation, and Kubernetes network policies restricting pod-to-pod traffic between tenant-scoped worker queues in high-tier dedicated deployments.

7A.2 Secrets Management

All service credentials, including database connection strings and OpenAI API keys for opt-in tenants, are stored in Azure Key Vault or AWS Secrets Manager and injected into Kubernetes pods via CSI Secret Store driver, never as plain environment variables in Helm values files, consistent with the Coding Standards in Engineering Standards (STL-ENG-STD-003), Section 2.1.

7A.3 Error Handling Strategy

Services follow the standard error envelope defined in Engineering Standards, Section 8.3. Retrieval and Generation services additionally implement circuit breakers around external LLM API calls; if the OpenAI API error rate exceeds 10% over a 1-minute window, the circuit breaker opens and the platform falls back to the tenant’s configured on-prem Ollama backend where available, or returns a graceful degraded-service message.

7B. Caching Strategy

7B.1 Query Result Caching

Frequently repeated queries within a tenant (identified via normalized query hashing) are cached in Redis with a 15-minute TTL, reducing redundant retrieval and generation calls for common questions such as “What is our leave policy?” This cache is invalidated automatically whenever the underlying tenant’s document set changes.

7B.2 Embedding Cache

Embeddings for unchanged document chunks are not regenerated on re-ingestion; the Ingestion Service computes a content hash per chunk and skips embedding generation when the hash matches an existing entry, directly supporting FR-106 in the Requirements Specification.

8. Internal Contacts

Purpose	Contact
Project Alpha Tech Lead	alpha-techlead@seriestech.com
Architecture Review Board	architecture-review@seriestech.com
DevOps On-Call (Alpha)	alpha-oncall@seriestech.com

9. Related Documents

- STL-ALPHA-REQ-004 — Project Alpha Requirements Specification
- STL-ALPHA-UM-006 — Project Alpha User Manual
- STL-ARCH-SYS-009 — Enterprise System Architecture & FAQ
- STL-ENG-STD-003 — Engineering Standards

10. References

- LangChain Retrieval Chain Architecture Notes, internal engineering wiki
- ChromaDB Collection Isolation Guidance, Architecture internal notes
- Ollama GPU Sizing Benchmarks, DevOps internal wiki, 2026

Internal Use Only — Series Tech Ltd. © 2026. Unauthorized distribution of this document outside the organization is prohibited.